

LAPORAN PRATIKUM

STRUKTUR DATA

Algoritma Sorting dan GUI

disusun Oleh:

Muhammad Yasin Habiburrahman

2511532016

Dosen Pengampu:

Dr. Wahyudi. S.T.M.T

Asisten Pratikum:

Sherly Sukmadira Putri



DEPARTEMEN INFORMATIKA FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

TAHUN 2026

KATA PENGANTAR

Puji dan syukur kami panjatkan ke hadirat Allah SWT atas segala rahmat dan karunia-Nya, sehingga kami dapat menyelesaikan tugas kelompok ini dengan baik. Shalawat serta salam semoga senantiasa tercurah kepada junjungan kita Nabi Muhammad SAW, keluarga, sahabat, serta para pengikutnya hingga akhir zaman.

Kami menyadari bahwa penyusunan tugas ini masih jauh dari sempurna, baik dari segi materi maupun penyajiannya. Oleh karena itu, kami sangat mengharapkan kritik dan saran yang membangun dari semua pihak demi kesempurnaan tugas kami di masa mendatang.

Akhir kata, kami mengucapkan terima kasih kepada dosen pembimbing yang telah memberikan arahan, serta kepada seluruh anggota kelompok yang telah bekerja sama dengan baik sehingga tugas ini dapat terselesaikan tepat waktu. Semoga tugas ini dapat bermanfaat bagi kami khususnya, dan bagi pembaca pada umumnya.

Padang, 20 Mei 2026

Muhammad Yasin Habiburrahman

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang	1
1.2 Tujuan	1
1.3 Manfaat Praktikum.....	1
BAB II PEMBAHASAN.....	2
2.1 Insertion Sort.....	2
2.2 Selection Sort	3
2.3 Bubble Sort	3
2.4 GUI Insertion Sort.....	4
2.4.1 Atribut	4
2.4.2 Konstruktor	5
2.4.3 Menetapkan Array dari Input.....	6
2.4.4 Menampilkan Langkah Langkah.....	7
2.4.5 Mengganti Visual.....	8
2.4.6 Mereset Program.....	8
2.4.7 Mengubah Elemen-Element Array menjadi String.....	9
2.4.8 Tampilan Program.....	9
BAB III LAPORAN TUGAS PRAKTIKUM.....	11
3.1 ADT Mahasiswa	11
3.2 Deklarasi Atribut-Atribut Kelas.....	11
3.3 GUI Mahasiswa	12
3.3.1 Properti Program	12
3.3.2 Panel Input	13
3.3.3 Panel Tengah.....	13
3.3.4 Panel Kontrol	14
3.4 Method Pengelolaan Data	14
3.4.1 Menambahkan Data	14
3.4.2 Menghapus Data	15
3.4.3 Menampilkan Tabel Data.....	15
3.5 Menjalankan Sorting.....	16
3.6 Algoritma-Algoritma Sorting.....	17

3.7 Method Konversi	18
3.8 Method Reset	18
BAB IV KESIMPULAN.....	19
DAFTAR PUSTAKA.....	20

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam pengolahan data, salah satu operasi yang paling sering dibutuhkan adalah pengurutan atau *sorting*. Sorting adalah proses menyusun elemen-elemen dalam suatu kumpulan data ke dalam urutan tertentu, baik ascending (dari kecil ke besar) maupun descending (dari besar ke kecil)

Terdapat berbagai algoritma sorting yang masing-masing memiliki cara kerja, kelebihan, dan keterbatasan tersendiri. Pada praktikum ini dipelajari tiga algoritma sorting dasar yaitu Bubble Sort, Selection Sort, dan Insertion Sort. Bubble Sort bekerja dengan cara membandingkan dua elemen yang berdekatan dan menukarnya jika urutannya salah, berulang hingga seluruh data terurut. Selection Sort bekerja dengan cara mencari elemen terkecil dari bagian yang belum terurut lalu menempatkannya di posisi yang benar. Insertion Sort bekerja dengan cara mengambil satu elemen dan menyisipkannya ke posisi yang tepat di antara elemen-elemen yang sudah terurut sebelumnya.

1.2 Tujuan

1. Memahami konsep dan cara kerja algoritma Bubble Sort, Selection Sort, dan Insertion Sort.
2. Mampu mengimplementasikan ketiga algoritma sorting tersebut menggunakan bahasa pemrograman Java.
3. Mampu membangun aplikasi GUI sederhana menggunakan Java Swing untuk memvisualisasikan proses sorting secara langkah per langkah.

1.3 Manfaat Praktikum

- Praktikan mampu menerapkan algoritma sorting dasar dalam menyelesaikan permasalahan pemrograman yang membutuhkan pengurutan data.
- Praktikan memiliki pemahaman yang lebih baik mengenai perbedaan efisiensi antar algoritma sorting, sebagai dasar untuk mempelajari algoritma sorting yang lebih efisien seperti Merge Sort dan Quick Sort.
- Praktikan mendapatkan pengalaman membangun aplikasi GUI menggunakan Java Swing, yang memperluas kemampuan pemrograman dari berbasis teks (console) menjadi berbasis antarmuka grafis.

BAB II

PEMBAHASAN

2.1 Insertion Sort

```
public class InsertionSort_2511532016 {  
  
    public static void insertionSort_2016(int[] arr_2016) {  
        int n_2016 = arr_2016.length;  
        for(int i_2016=1; i_2016<n_2016; i_2016++) {  
            int key_2016 = arr_2016[i_2016];  
            int j_2016 = i_2016-1;  
            while(j_2016>= 0 && arr_2016[j_2016]>key_2016) {  
                arr_2016[j_2016+1] = arr_2016[j_2016];  
                j_2016--;  
            }  
            arr_2016[j_2016+1] = key_2016;  
        }  
    }  
}
```

Kode Program 2.1 – Program Insertion Sort

Insertion sort bekerja dengan cara memasukkan elemen satu per satu ke dalam sub-array yang sudah terurut. Insertion sort dimulai dari indeks paling pertama, anggap indeks 1, yang dianggap sudah terurut karena hanya ada 1 elemen saja, kemudian bertambah menjadi 2 elemen dengan ditambah dengan elemen berikutnya. Elemen yang baru ditambahkan akan dibandingkan dengan elemen-elemen pada sub-array yang terurut dan akan meletakkannya pada posisi yang sesuai. Proses ini dilakukan sampai indeks terakhir pada array.

Langkah	Key	Hasil
1	78	23, 78, 45, 832, 56, 1
2	45	23, 45, 78, 832, 56, 1
3	832	23, 45, 78, 832, 56, 1
4	56	23, 45, 56, 78, 832, 1
5	1	1, 23, 45, 56, 78, 832

Tabel 2.1 – Simulasi Langkah-Langkah Sederhana Insertion Sort

2.2 Selection Sort

```
public class SelectionSort_2511532016 {
    public static void SelectionSort_2016(int[] arr_2016) {
        int n_2016 = arr_2016.length;
        for(int i_2016=0; i_2016<n_2016; i_2016++) {
            int minIndex_2016 = i_2016;
            for(int j_2016=i_2016+1; j_2016<n_2016; j_2016++) {
                if(arr_2016[j_2016] < arr_2016[minIndex_2016]) {
                    minIndex_2016 = j_2016;
                }
            }
            int temp_2016 = arr_2016[i_2016];
            arr_2016[i_2016] = arr_2016[minIndex_2016];
            arr_2016[minIndex_2016] = temp_2016;
        }
    }
}
```

Kode Proram 2.2 – Program Selection Sort

Selection Sort bekerja dengan cara menggunakan pointer yang menyimpan nilai minimum. Mula-mula ia akan melakukan iterasi dari elemen pertama sampai elemen terakhir sekaligus melakukan perbandingan elemen. Elemen yang dibandingkan adalah elemen dengan nilai paling kecil. Pada iterasi pertama, elemen paling kecil akan diletakkan pada posisi pertama. Pada iterasi kedua, elemen terkecil kedua akan diletakkan pada posisi kedua, dan seterusnya

Langkah	Min	Hasil
1	1	1, 78, 45, 832, 56, 23
2	23	1, 23, 45, 832, 56, 78
3	45	1, 23, 45, 832, 56, 78
4	56	1, 23, 45, 56, 832, 78
5	78	1, 23, 45, 56, 78, 832

Tabel 2.2 – Simulasi Langkah-Langkah Sederhana Selection Sort

2.3 Bubble Sort

```
public static void BubbleSort_2016(int[] arr_2016) {
    int n_2016 = arr_2016.length;
    for(int i_2016=0; i_2016<n_2016;i_2016++) {
        for(int j_2016=0; j_2016<n_2016-i_2016-1;j_2016++ ) {
            if(arr_2016[j_2016] > arr_2016[j_2016+1]) {
                int temp_2016 = arr_2016[j_2016];
                arr_2016[j_2016] = arr_2016[j_2016+1];
                arr_2016[j_2016+1] = temp_2016;
            }
        }
    }
}
```

Kode Program 2.3 – Program Bubble Sort

Bubble sort bekerja dengan cara meletakkan elemen paling tinggi pada posisi paling akhir dari sub-array terurut. Mirip dengan selection sort yang meletakkan elemen paling kecil pada sub-array terurut dari indeks 1, bubble sort melakukannya untuk elemen dengan nilai paling tinggi. Caranya adalah dengan melakukan iterasi dari indeks paling awal hingga 1 posisi sebelum sub-array yang sudah terurut pada setiap iterasi. Untuk setiap iterasi, ia akan menggerakkan elemen paling besar yang ditemukan. Misalkan ukuran array adalah N. Pada iterasi pertama, elemen paling tinggi diletakkan pada indeks ke N-1. Elemen terbesar kedua akan diletakkan pada posisi N-2, dan seterusnya.

Langkah	Hasil
1	23, 45, 78, 56, 1, 832
2	23, 45, 56, 1, 78 , 832
3	23, 45, 1, 56 , 78, 832
4	23, 1, 45 , 56, 78, 832
5	1 , 23, 45, 56, 78, 832

Tabel 2.3 - Simulasi Langkah-Langkah Sederhana Bubble Sort

2.4 GUI Insertion Sort

2.4.1 Atribut

```
private int[] array_2016;
private JLabel[] labelArray_2016;
private JButton stepButton_2016, resetButton_2016, setButton_2016;
private JTextField inputField_2016;
private JPanel panelArray_2016;
private JTextArea stepArea_2016;

private int i_2016 =1, j_2016;
private boolean sorting_2016 = false;
private int stepCount_2016 =1;
```

Kode Program 2.4 – Atribut Program GUI Insertion Sort

array yang menyimpan data angka yang akan diurutkan. labelArray bertipe JLabel[] adalah array komponen visual. Setiap elemennya merepresentasikan satu kotak angka yang tampil di layar, jumlahnya selalu sama dengan array.

Untuk komponen GUI yaitu stepButton, resetButton, dan setButton bertipe JButton adalah tombol-tombol interaktif. inputField bertipe JTextField adalah kotak teks tempat pengguna memasukkan angka. panelArray bertipe JPanel adalah wadah untuk menampilkan kotak-kotak angka secara visual, dan stepArea bertipe JTextArea adalah area teks multi-baris untuk menampilkan log langkah sorting.

Variabel kontrol terdapat i yang bertipe int diinisialisasi ke 1 sebagai indeks loop luar insertion sort (dimulai dari 1 karena elemen pertama dianggap sudah terurut). j bertipe int adalah indeks loop dalam untuk menggeser elemen. sorting bertipe

boolean adalah flag yang menandai apakah proses sorting sedang berjalan. stepCount bertipe int diinisialisasi ke 1 sebagai penghitung nomor langkah yang ditampilkan di log.

2.4.2 Konstruktor

```
public InsertionSort_GUI2511532016() {
    setTitle("Insertion Sort Langkah per Langkah");
    setSize(750, 400);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setLocationRelativeTo(null);
    setLayout(new BorderLayout());

    //Panel Input
    JPanel inputPanel_2016 = new JPanel(new FlowLayout());
    inputField_2016 = new JTextField(30);
    setButton_2016 = new JButton("Set Array");
    inputPanel_2016.add(new JLabel("Masukkan angka (pisahkan dengan koma)"));
    inputPanel_2016.add(inputField_2016);
    inputPanel_2016.add(setButton_2016);

    //Panel Array visual
    panelArray_2016 = new JPanel();
    panelArray_2016.setLayout(new FlowLayout());

    //Panel Kontrol
    JPanel controlPanel_2016 = new JPanel();
    stepButton_2016 = new JButton("Langkah selanjutnya");
    resetButton_2016 = new JButton("Reset");
    stepButton_2016.setEnabled(false);
    controlPanel_2016.add(stepButton_2016);
    controlPanel_2016.add(resetButton_2016);

    //Area teks untuk log langkah-langkah
    stepArea_2016 = new JTextArea(8,60);
    stepArea_2016.setEditable(false);
    stepArea_2016.setFont(new Font("Monospaced", Font.PLAIN, 14));
    JScrollPane scrollPane_2016 = new JScrollPane(stepArea_2016);

    //Tambahkan panel ke frame
    add(inputPanel_2016, BorderLayout.NORTH);
    add(panelArray_2016, BorderLayout.CENTER);
    add(controlPanel_2016, BorderLayout.SOUTH);
    add(scrollPane_2016, BorderLayout.EAST);

    //Event set array
    setButton_2016.addActionListener( e-> setArrayFromInput_2016());

    //Event set array
    stepButton_2016.addActionListener( e-> performStep());

    //Event set array
    resetButton_2016.addActionListener( e-> reset_2016());
}
```

Kode Program 2.5 – Konstruktor GUI Insertion Sort

Bagian pertama mengatur properti dasar jendela. `setTitle()` memberi judul pada title bar, `setSize(750, 400)` mengatur ukuran jendela dalam piksel. `setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE)` memastikan program benar-benar berhenti saat jendela ditutup, `setLocationRelativeTo(null)` membuat jendela muncul di tengah layar, dan `setLayout(new BorderLayout())` mengatur tata letak jendela menjadi 5 region, NORTH, SOUTH, EAST, WEST, dan CENTER.

`inputPanel` menggunakan `FlowLayout` yang menyusun komponen secara horizontal. Berisi label instruksi, text field berkapasitas 30 karakter, dan tombol "Set Array". Panel ini ditempatkan di NORTH.

`panelArray` juga menggunakan `FlowLayout` sebagai wadah kotak-kotak angka visual. Isinya akan diisi secara dinamis saat array di-set. Ditempatkan di CENTER.

`controlPanel` berisi dua tombol, yaitu "Langkah selanjutnya" dan "Reset". Tombol step dinonaktifkan dulu dengan `setEnabled(false)` karena belum ada array yang di-set. Ditempatkan di SOUTH.

`scrollPane` membungkus `stepArea` agar bisa di-*scroll* jika log langkah terlalu panjang. `stepArea` diatur `setEditable(false)` agar pengguna tidak bisa mengedit log, dan menggunakan font `Monospaced` agar teks log rapi seperti output terminal. Ditempatkan di EAST.

Terakhir, tiga event listener ditambahkan ke masing-masing tombol menggunakan lambda expression — `setButton` memanggil `setArrayFromInput()`, `stepButton` memanggil `performStep()`, dan `resetButton` memanggil `reset()`. Perlu dicatat ketiga komentar event listener di kode bertuliskan "Event set array" semua — ini typo dari dosennya, seharusnya masing-masing berbeda sesuai tombolnya.

2.4.3 Menetapkan Array dari Input

```
private void setArrayFromInput_2016() {
    String text_2016 = inputField_2016.getText().trim();
    if(text_2016.isEmpty()) return;
    String[] parts_2016 = text_2016.split(",");
    array_2016 = new int[parts_2016.length];

    try {
        for(int k_2016=0; k_2016< parts_2016.length; k_2016++) {
            array_2016[k_2016] = Integer.parseInt(parts_2016[k_2016].trim());
        } catch (NumberFormatException e) {
            JOptionPane.showMessageDialog(this, "Masukan hanya angka yang dipisahkan dengan koma!", "Error", JOptionPane.ERROR_MESSAGE);
            return;
        }
    }
    i_2016=1;
    stepCount_2016 = 1;
    sorting_2016 = true;
    stepButton_2016.setEnabled(true);
    stepArea_2016.setText("");
    panelArray_2016.removeAll();
    labelArray_2016 = new JLabel[array_2016.length];
    for(int k_2016=0; k_2016<array_2016.length; k_2016++) {
        labelArray_2016[k_2016] = new JLabel(String.valueOf(array_2016[k_2016]));
        labelArray_2016[k_2016].setFont(new Font("Arial", Font.BOLD, 24));
        labelArray_2016[k_2016].setBorder(BorderFactory.createLineBorder(Color.black));
        labelArray_2016[k_2016].setPreferredSize(new Dimension(50,50));
        labelArray_2016[k_2016].setHorizontalAlignment(SwingConstants.CENTER);
        panelArray_2016.add(labelArray_2016[k_2016]);
    }
    panelArray_2016.revalidate();
    panelArray_2016.repaint();
}
```

Kode Program 2.6 – Program untuk Menetapkan Sebuah Array dari Input Pengguna

Method ini dipanggil saat tombol "Set Array" ditekan. Pertama, teks dari inputField dibaca dan di-trim untuk menghapus spasi di ujung. Jika field kosong, method langsung berhenti dengan return.

Jika ada isinya, teks dipecah berdasarkan karakter koma menggunakan `split(",")` menghasilkan array String parts. Array baru kemudian dibuat dengan panjang sesuai jumlah elemen yang diinput. Proses konversi String ke int dilakukan dalam blok try-catch — setiap elemen di-trim lagi untuk menghapus spasi yang mungkin ada di antara koma, lalu dikonversi menggunakan `Integer.parseInt()`. Jika ada karakter non-angka, `NumberFormatException` ditangkap dan ditampilkan dialog error via `JOptionPane`, lalu method berhenti.

Jika konversi berhasil, variabel sorting direset i kembali ke 1, `stepCount` ke 1, sorting di-set true, dan tombol step diaktifkan. Log dikosongkan dan semua komponen lama di `panelArray` dihapus dengan `removeAll()`.

Kemudian `labelArray` dibuat baru dengan panjang sama seperti array. Setiap elemen dibuatkan `JLabel` dengan nilai awalnya, diatur font Arial Bold ukuran 24, diberi border hitam, ukuran fixed 50x50 piksel, dan teks di-center menggunakan `SwingConstants.CENTER`. Setiap label ditambahkan ke `panelArray`. Terakhir `revalidate()` dan `repaint()` dipanggil agar perubahan tampil di layar — `revalidate()` menghitung ulang layout, `repaint()` menggambar ulang komponen.

2.4.4 Menampilkan Langkah Langkah

```
private void performStep() {
    if(i_2016 < array_2016.length && sorting_2016) {
        int key_2016 = array_2016[i_2016];
        j_2016 = i_2016-1;

        StringBuilder stepLog_2016 = new StringBuilder();
        stepLog_2016.append("Langkah ").append(stepCount_2016).append("\n").append(": Memasukkan ").append(key_2016).append("\n");

        while(j_2016 >= 0 && array_2016[j_2016] > key_2016) {
            array_2016[j_2016+1] = array_2016[j_2016];
            j_2016--;
        }
        array_2016[j_2016+1] = key_2016;

        updateLabels_2016();
        stepLog_2016.append("Hasil: ").append(arrayToString_2016(array_2016)).append("\n\n");
        stepArea_2016.append(stepLog_2016.toString());

        i_2016++;
        stepCount_2016++;

        if(i_2016 == array_2016.length) {
            sorting_2016 = false;
            stepButton_2016.setEnabled(false);
            JOptionPane.showMessageDialog(this, "Sorting Selesai");
        }
    }
}

private void updateLabels_2016() {
    for(int k_2016=0; k_2016<array_2016.length; k_2016++) {
        labelArray_2016[k_2016].setText(String.valueOf(array_2016[k_2016]));
    }
}
```

Kode Program 2.7 – Fungsi yang Digunakan untuk Menampilkan Tiap Langkah saat Tombol Ditekan

Method ini dipanggil setiap kali tombol "Langkah selanjutnya" ditekan, menjalankan satu iterasi insertion sort per klik.

Pertama dicek dua kondisi; $i < \text{array.length}$ memastikan masih ada elemen yang belum diproses, dan $\text{sorting} == \text{true}$ memastikan sorting masih berjalan. Jika salah satu tidak terpenuhi, method tidak melakukan apapun.

Jika kondisi terpenuhi, key diambil dari $\text{array}[i]$. Ini elemen yang akan disisipkan ke posisi yang benar. j diinisialisasi ke $i-1$ sebagai indeks pembanding di sebelah kiri key.

Kemudian `StringBuilder` dibuat untuk menyusun log langkah. Log menulis `stepCount` sebagai nomor langkah.

Loop while kemudian menggeser semua elemen yang lebih besar dari key ke kanan satu posisi dengan $\text{array}[j+1] = \text{array}[j]$, sambil terus mendekremen nilai j . Loop berhenti saat $j < 0$ atau elemen di kiri sudah lebih kecil dari key. Setelah loop, key ditempatkan di $\text{array}[j+1]$.

2.4.5 Mengganti Visual

```
private void updateLabels_2016() {
    for(int k_2016=0; k_2016<array_2016.length; k_2016++) {
        labelArray_2016[k_2016].setText(String.valueOf(array_2016[k_2016]));
    }
}
```

Kode Program 2.8 – Fungsi untuk Mengubah Tampilan saat Tombol Ditekan

Setelah array diperbarui, `updateLabels()` dipanggil untuk menyinkronkan tampilan visual dengan data terbaru, lalu hasil array ditambahkan ke log menggunakan `arrayToString()`, dan log di-append ke `stepArea`.

Terakhir i dan `stepCount` dinaikkan. Jika i sudah sama dengan panjang array, berarti semua elemen sudah diproses. Sorting di-set menjadi false, tombol `step` dinonaktifkan, dan dialog "Sorting Selesai" ditampilkan via `JOptionPane`.

2.4.6 Mereset Program

```
private void reset_2016() {
    inputField_2016.setText("");
    panelArray_2016.removeAll();
    panelArray_2016.revalidate();
    panelArray_2016.repaint();
    stepArea_2016.setText("");
    stepButton_2016.setEnabled(false);
    sorting_2016 = false;
    i_2016=1;
    stepCount_2016 = 1;
}
```

Kode Program 2.9 – Fungsi untuk Mereset Program saat Tombol Ditekan

Method ini dipanggil saat tombol "Reset" ditekan, mengembalikan seluruh kondisi program ke keadaan awal. inputField dikosongkan, semua komponen visual di panelArray dihapus dengan removeAll(), lalu revalidate() dan repaint() dipanggil untuk memperbarui tampilan. Log di stepArea dikosongkan, tombol step dinonaktifkan, flag sorting di-set false, dan kedua counter i serta stepCount dikembalikan ke 1. Setelah reset, program siap menerima input array baru dari pengguna.

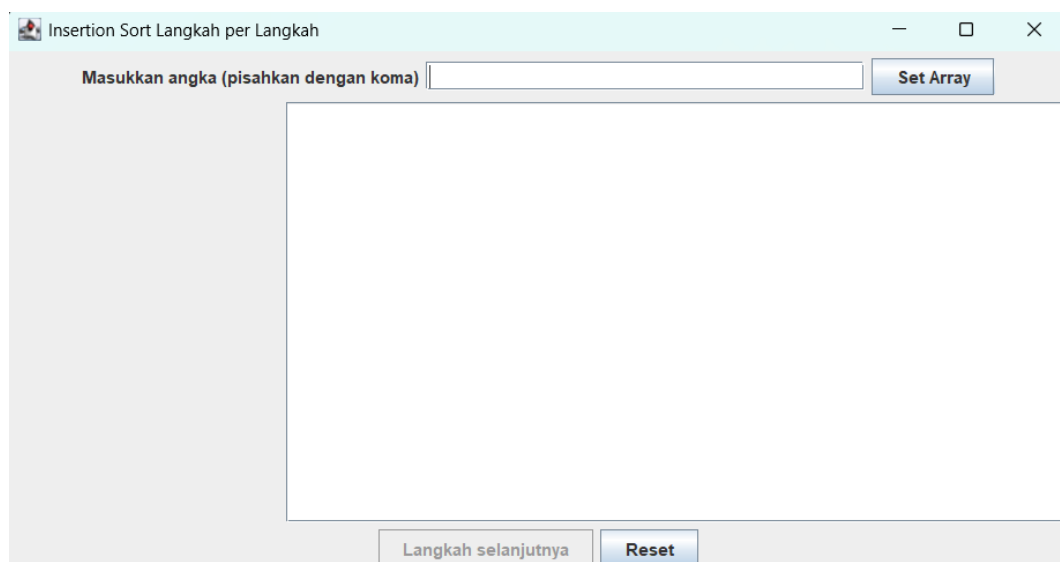
2.4.7 Mengubah Elemen-Elemen Array menjadi String

```
private String arrayToString_2016(int[] arr_2016) {
    StringBuilder sb_2016 = new StringBuilder();
    for(int k_2016=0; k_2016< arr_2016.length; k_2016++) {
        sb_2016.append(arr_2016[k_2016]);
        if(k_2016<arr_2016.length -1) sb_2016.append(", ");
    }
    return sb_2016.toString();
}
```

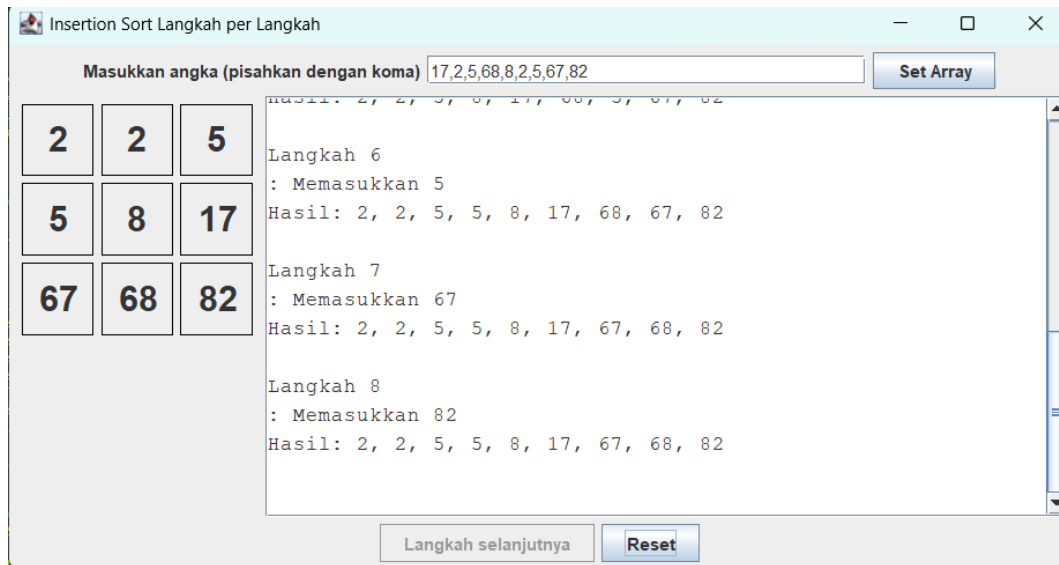
Kode Program 2.10 – Fungsi untuk Mengubah Elemen Array menjadi String

Method helper yang mengkonversi array int[] menjadi String berformat 1, 2, 3, 4 untuk keperluan log. Menggunakan StringBuilder agar penggabungan String lebih efisien dibanding operator + biasa. Setiap elemen di-append satu per satu, dan koma ditambahkan di antara elemen menggunakan kondisi $k < \text{arr.length} - 1$. Hasilnya dikembalikan via toString() dan dipakai oleh performStep() untuk mencetak kondisi array di setiap langkah.

2.4.8 Tampilan Program



Gambar 2.1 – Tampilan saat Program Dijalankan



Gambar 2.2 – Contoh Sampel saat Program Dijalankan

BAB III

LAPORAN TUGAS PRAKTIKUM

3.1 ADT Mahasiswa

```
//ADT Mahasiswa - menyimpan data seorang mahasiswa
class Mahasiswa_2511532016 {
    // Atribut
    private String nama_2016;
    private String nim_2016;
    private String prodi_2016;

    // Constructor
    public Mahasiswa_2511532016(String nama_2016, String nim_2016, String prodi_2016) {
        this.nama_2016 = nama_2016;
        this.nim_2016 = nim_2016;
        this.prodi_2016 = prodi_2016;
    }

    // Getter
    public String getNama_2016() { return nama_2016; }
    public String getNim_2016() { return nim_2016; }
    public String getProdi_2016(){ return prodi_2016; }

    // Setter
    public void setNama_2016(String nama_2016) { this.nama_2016 = nama_2016; }
    public void setNim_2016(String nim_2016) { this.nim_2016 = nim_2016; }
    public void setProdi_2016(String prodi_2016) { this.prodi_2016 = prodi_2016; }

    @Override
    public String toString() {
        return nama_2016 + " | " + nim_2016 + " | " + prodi_2016;
    }
}
```

Kode Program 3.1 – Blok Program ADT Mahasiswa

ADT ini memiliki atribut-atribut yang dimiliki oleh mahasiswa, yaitu nama, nim, dan prodi. Terdapat juga konstruktor untuk membuat objek mahasiswa, beserta setter dan getter untuk masing-masing atribut. Terdapat pula fungsi override toString, yang dapat digunakan untuk menampilkan nama, nim, dan prodi mahasiswa.

3.2 Deklarasi Atribut-Atribut Kelas

```
// Data
private ArrayList<Mahasiswa_2511532016> dataMahasiswa_2016 = new ArrayList<>();

// Komponen input
private JTextField fieldNama_2016, fieldNim_2016, fieldProdi_2016;
private JButton btnTambah_2016, btnHapus_2016, btnSorting_2016, btnReset_2016;
private JComboBox<String> comboAlgoritma_2016;

// Area tampilan
private JTextArea areaTabelData_2016;
private JTextArea areaHasil_2016;
```

Kode Program 3.2 – Atribut-Atribut pada GUI

Atribut class dibagi menjadi tiga kelompok. Pertama, `dataMahasiswa_2016` bertipe `ArrayList<Mahasiswa_2511532016>` sebagai penyimpan seluruh data mahasiswa yang diinput menggunakan generic type parameter `Mahasiswa_2511532016` agar hanya bisa menyimpan objek bertipe tersebut tanpa perlu casting saat pengambilan data.

Kedua, komponen input GUI. `fieldNama_2016`, `fieldNim_2016`, dan `fieldProdi_2016` bertipe `TextField` sebagai kotak teks untuk memasukkan nama, NIM, dan program studi. `btnTambah_2016`, `btnHapus_2016`, `btnSorting_2016`, dan `btnReset_2016` bertipe `JButton` sebagai tombol-tombol interaktif. `comboAlgoritma_2016` bertipe `JComboBox<String>` sebagai dropdown untuk memilih algoritma sorting yang akan dijalankan.

Ketiga, area tampilan `areaTabelData_2016` bertipe `JTextArea` untuk menampilkan daftar data mahasiswa yang sudah diinput, dan `areaHasil_2016` bertipe `JTextArea` untuk menampilkan visualisasi proses sorting langkah per langkah beserta hasil akhirnya.

3.3 GUI Mahasiswa

3.3.1 Properti Program

```
//Properti Program
setTitle("Pengurutan Nama Mahasiswa - Sorting Algoritma");
setSize(850, 600);
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
setLocationRelativeTo(null);
setLayout(new BorderLayout(10, 10));
```

Kode Program 3.1 – Blok Program untuk Identitas Awal Program

Bagian ini adalah inisialisasi properti dasar jendela di dalam konstruktor. `setTitle()` mengatur teks yang muncul di title bar jendela. `setSize(850, 600)` mengatur lebar dan tinggi jendela dalam satuan piksel. `setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE)` memastikan seluruh program benar-benar berhenti saat tombol close (X) ditekan. Tanpa baris ini, jendela tertutup tapi proses Java masih berjalan di background. `setLocationRelativeTo(null)` membuat jendela muncul di tengah layar saat pertama kali dijalankan. `setLayout(new BorderLayout(10, 10))` mengatur tata letak jendela menggunakan `BorderLayout` dengan jarak horizontal dan vertikal antar komponen sebesar 10 piksel.

3.3.2 Panel Input

```
// ===== PANEL INPUT (NORTH) =====
JPanel panelInput_2016 = new JPanel(new GridLayout(4, 2, 5, 5));
panelInput_2016.setBorder(BorderFactory.createTitledBorder("Input Data Mahasiswa"));

fieldNama_2016 = new JTextField();
fieldNim_2016 = new JTextField();
fieldProdi_2016 = new JTextField();

panelInput_2016.add(new JLabel("Nama Mahasiswa :"));
panelInput_2016.add(fieldNama_2016);
panelInput_2016.add(new JLabel("NIM :"));
panelInput_2016.add(fieldNim_2016);
panelInput_2016.add(new JLabel("Program Studi :"));
panelInput_2016.add(fieldProdi_2016);

// Tombol tambah & hapus
JPanel panelTombolInput_2016 = new JPanel(new FlowLayout(FlowLayout.LEFT));
btnTambah_2016 = new JButton("Tambah Data");
btnHapus_2016 = new JButton("Hapus Data Terakhir");
panelTombolInput_2016.add(btnTambah_2016);
panelTombolInput_2016.add(btnHapus_2016);
panelInput_2016.add(panelTombolInput_2016);
```

Kode Program 3.2 – Blok Program Pengatur Input

Panel ini terletak pada bagian atas pada layout. Panel ini memiliki field untuk mengisi nama, nim, dan prodi dari pengguna.

panelTombolInput tersendiri menggunakan FlowLayout(FlowLayout.LEFT) yang menyusun komponen secara horizontal rata kiri. Panel ini menampung tombol "Tambah Data" dan "Hapus Data Terakhir", lalu panel tombol tersebut ditambahkan sebagai satu komponen ke panelInput_2016.

3.3.3 Panel Tengah

```
// ===== PANEL TENGAH (CENTER) =====
JPanel panelTengah_2016 = new JPanel(new GridLayout(1, 2, 10, 0));

// Tabel data mahasiswa
areaTabelData_2016 = new JTextArea();
areaTabelData_2016.setEditable(false);
areaTabelData_2016.setFont(new Font("Monospaced", Font.PLAIN, 13));
JScrollPane scrollTabel_2016 = new JScrollPane(areaTabelData_2016);
scrollTabel_2016.setBorder(BorderFactory.createTitledBorder("Data Mahasiswa"));

// Area hasil sorting
areaHasil_2016 = new JTextArea();
areaHasil_2016.setEditable(false);
areaHasil_2016.setFont(new Font("Monospaced", Font.PLAIN, 13));
JScrollPane scrollHasil_2016 = new JScrollPane(areaHasil_2016);
scrollHasil_2016.setBorder(BorderFactory.createTitledBorder("Visualisasi & Hasil Sorting"));

panelTengah_2016.add(scrollTabel_2016);
panelTengah_2016.add(scrollHasil_2016);
```

Kode Program 3.3 – Blok Program Panel Tengah

Pada bagian tengah program, kolom kiri berisi areaTabelData_2016 yang menampilkan daftar data mahasiswa. Kolom kanan berisi areaHasil_2016 yang menampilkan visualisasi proses sorting dan hasil akhirnya. setFont(new

Font("Monospaced", Font.PLAIN, 13)) menggunakan font monospaced agar kolom-kolom teks terformat rapi karena setiap karakter memiliki lebar yang sama.

3.3.4 Panel Kontrol

```
// ===== PANEL KONTROL (SOUTH) =====
JPanel panelKontrol_2016 = new JPanel(new FlowLayout());
panelKontrol_2016.setBorder(BorderFactory.createTitledBorder("Pilih & Jalankan Sorting"));

String[] algoritma_2016 = {"Insertion Sort", "Selection Sort", "Bubble Sort"};
JComboBox<> comboAlgoritma_2016 = new JComboBox<>(algoritma_2016);
JButton btnSorting_2016 = new JButton("Mulai Sorting");
JButton btnReset_2016 = new JButton("Reset");

panelKontrol_2016.add(new JLabel("Algoritma : "));
panelKontrol_2016.add(comboAlgoritma_2016);
panelKontrol_2016.add(btnSorting_2016);
panelKontrol_2016.add(btnReset_2016);

// ===== TAMBAHKAN KE FRAME =====
add(panelInput_2016, BorderLayout.NORTH);
add(panelTengah_2016, BorderLayout.CENTER);
add(panelKontrol_2016, BorderLayout.SOUTH);

// ===== EVENT LISTENER =====
btnTambah_2016.addActionListener(e -> tambahData_2016());
btnHapus_2016.addActionListener(e -> hapusData_2016());
btnSorting_2016.addActionListener(e -> jalankanSorting_2016());
btnReset_2016.addActionListener(e -> reset_2016());
```

Kode Program 3.4 – Blok Program Panel Kontrol

panelKontrol_2016 menggunakan FlowLayout yang menyusun komponen secara horizontal di tengah.

comboAlgoritma_2016 dibuat dari array String algoritma_2016 yang berisi tiga pilihan, yaitu "Insertion Sort", "Selection Sort", dan "Bubble Sort" yang langsung dioper ke konstruktor JComboBox.

Pengguna cukup klik dropdown untuk memilih algoritma yang diinginkan sebelum sorting dijalankan. btnSorting_2016 dan btnReset_2016 dibuat sebagai tombol untuk memulai sorting dan mereset program.

Terakhir, seluruh tiga panel utama ditambahkan ke frame sesuai posisinya masing-masing. panelInput_2016 di NORTH, panelTengah_2016 di CENTER, dan panelKontrol_2016 di SOUTH.

3.4 Method Pengelolaan Data

3.4.1 Menambahkan Data

```
// ===== TAMPIL DATA =====
private void tambahData_2016() {
    String nama_2016 = fieldNama_2016.getText().trim();
    String nim_2016 = fieldNim_2016.getText().trim();
    String prodi_2016 = fieldProdi_2016.getText().trim();

    if (nama_2016.isEmpty() || nim_2016.isEmpty() || prodi_2016.isEmpty()) {
        JOptionPane.showMessageDialog(this, "Semua field harus diisi!", "Error", JOptionPane.ERROR_MESSAGE);
        return;
    }

    dataMahasiswa_2016.add(new Mahasiswa_2511532016(nama_2016, nim_2016, prodi_2016));
    fieldNama_2016.setText("");
    fieldNim_2016.setText("");
    fieldProdi_2016.setText("");
    tampilkanTabel_2016();
}
```

Kode Program 3.5 – Method Penambahan Data

Method ini dipanggil saat tombol "Tambah Data" ditekan. Pertama, nilai dari ketiga JTextField dibaca menggunakan `getText()` dan di-trim untuk menghapus spasi di ujung. Jika salah satu field kosong, ditampilkan dialog error menggunakan `JOptionPane.showMessageDialog()` dengan tipe `ERROR_MESSAGE` lalu method berhenti. Jika semua field terisi, objek `Mahasiswa_2511532016` baru dibuat langsung dari ketiga nilai tersebut dan ditambahkan ke `dataMahasiswa_2016` menggunakan `add()`. Setelah itu ketiga field dikosongkan kembali agar siap menerima input berikutnya, dan `tampilkanTabel_2016()` dipanggil untuk memperbarui tampilan.

3.4.2 Menghapus Data

```
// ===== HAPUS DATA TERAKHIR =====
private void hapusData_2016() {
    if (dataMahasiswa_2016.isEmpty()) {
        JOptionPane.showMessageDialog(this, "Data kosong!", "Info", JOptionPane.INFORMATION_MESSAGE);
        return;
    }
    dataMahasiswa_2016.remove(dataMahasiswa_2016.size() - 1);
    tampilkanTabel_2016();
}
```

Kode Program 3.6 – Method Penghapusan Data

Method ini menghapus elemen terakhir dari `dataMahasiswa_2016` menggunakan `remove(size()-1)`. Sebelum menghapus, dicek dulu apakah list kosong menggunakan `isEmpty()`. Jika kosong, ditampilkan dialog informasi dengan tipe `INFORMATION_MESSAGE` lalu method berhenti. Setelah penghapusan berhasil, `tampilkanTabel_2016()` dipanggil untuk memperbarui tampilan.

3.4.3 Menampilkan Tabel Data

```
// ===== TAMPILKAN TABEL DATA =====
private void tampilkanTabel_2016() {
    StringBuilder sb_2016 = new StringBuilder();
    sb_2016.append(String.format("%-5s %-20s %-15s %-20s\n", "No", "Nama", "NIM", "Prodi"));
    sb_2016.append("-".repeat(62)).append("\n");
    for (int i_2016 = 0; i_2016 < dataMahasiswa_2016.size(); i_2016++) {
        Mahasiswa_2511532016 mhs = dataMahasiswa_2016.get(i_2016);
        sb_2016.append(String.format("%-5d %-20s %-15s %-20s\n",
            i_2016 + 1,
            mhs.getNama_2016(),
            mhs.getNim_2016(),
            mhs.getProdi_2016()));
    }
    areaTabelData_2016.setText(sb_2016.toString());
}
```

Kode Program 3.7 – Method Penampil Data

Method ini membangun tampilan tabel data mahasiswa menggunakan `StringBuilder`. Header tabel dicetak menggunakan `String.format()` dengan format `%-5s %-20s %-15s %-20s`, dan angkanya menentukan lebar kolom dalam karakter. Garis pemisah dibuat menggunakan `"-".repeat(62)` sebagai pembatas antara header dan isi tabel. Kemudian setiap objek mahasiswa di list diiterasi dan dicetak dalam format yang sama menggunakan getter masing-masing atribut. Hasil akhir

StringBuilder di-set ke areaTabelData_2016 menggunakan setText() sehingga tampilan selalu mencerminkan kondisi data terkini.

3.5 Menjalankan Sorting

```
// ===== JALANKAN SORTING =====
private void jalankanSorting_2016() {
    if (dataMahasiswa_2016.size() < 2) {
        JOptionPane.showMessageDialog(this, "Minimal 2 data mahasiswa!", "Info", JOptionPane.INFORMATION_MESSAGE);
        return;
    }

    // Salin data agar data asli tidak berubah
    ArrayList<Mahasiswa_2511532016> dataSorting_2016 = new ArrayList<>();
    for (Mahasiswa_2511532016 mhs : dataMahasiswa_2016) {
        dataSorting_2016.add(new Mahasiswa_2511532016(mhs.getNama_2016(), mhs.getNim_2016(), mhs.getProdi_2016()));
    }

    String pilihan_2016 = (String) comboAlgoritma_2016.getSelectedItem();
    StringBuilder log_2016 = new StringBuilder();

    // Jalankan algoritma sesuai pilihan
    if (pilihan_2016.equals("Insertion Sort")) {
        insertionSort_2016(dataSorting_2016, log_2016);
    } else if (pilihan_2016.equals("Selection Sort")) {
        selectionSort_2016(dataSorting_2016, log_2016);
    } else {
        bubbleSort_2016(dataSorting_2016, log_2016);
    }

    // Tampilkan hasil akhir
    log_2016.append("\n=== HASIL AKHIR (A-Z) ===\n");
    for (int i_2016 = 0; i_2016 < dataSorting_2016.size(); i_2016++) {
        log_2016.append((i_2016 + 1) + ". " + dataSorting_2016.get(i_2016).getNama_2016() + "\n");
    }

    areaHasil_2016.setText(log_2016.toString());
}
```

Kode Program 3.8 – Method untuk Menjalankan Pengurutan(Sorting)

Method ini dipanggil saat tombol "Mulai Sorting" ditekan. Pertama dicek apakah jumlah data kurang dari 2. Jika iya ditampilkan dialog informasi dan method berhenti, karena sorting tidak bermakna jika data hanya satu atau kosong.

Sebelum sorting dijalankan, data asli disalin terlebih dahulu ke ArrayList baru dataSorting_2016 menggunakan perulangan yang membuat objek Mahasiswa_2511532016 baru untuk setiap elemennya.

Selanjutnya pilihan algoritma dibaca dari comboAlgoritma_2016 menggunakan getItem() yang dikasting ke String. Berdasarkan nilai tersebut, method sorting yang sesuai dipanggil. insertionSort_2016(), selectionSort_2016(), atau bubbleSort_2016(), dengan mengoper dataSorting_2016 dan log_2016 sebagai parameter. StringBuilder log_2016 dioper sebagai parameter agar setiap method sorting bisa menulis log visualisasinya langsung ke objek yang sama.

Setelah sorting selesai, hasil akhir ditambahkan ke log dengan mencetak seluruh nama mahasiswa dalam urutan yang sudah terurut secara ascending (A-Z). Keseluruhan log kemudian ditampilkan di areaHasil_2016 menggunakan setText().

3.6 Algoritma-Algorithm Sorting

```
// ===== INSERTION SORT =====
private void insertionSort_2016(ArrayList<Mahasiswa_2511532016> data_2016, StringBuilder log_2016) {
    log_2016.append("=== INSERTION SORT ===\n\n");
    int n_2016 = data_2016.size();

    for (int i_2016 = 1; i_2016 < n_2016; i_2016++) {
        // Simpan elemen yang akan disisipkan
        Mahasiswa_2511532016 key_2016 = data_2016.get(i_2016);
        int j_2016 = i_2016 - 1;

        // Geser elemen yang lebih besar ke kanan
        while (j_2016 >= 0 && data_2016.get(j_2016).getNama_2016().compareToIgnoreCase(key_2016.getNama_2016()) > 0) {
            data_2016.set(j_2016 + 1, data_2016.get(j_2016));
            j_2016--;
        }
        // Sisipkan key ke posisi yang tepat
        data_2016.set(j_2016 + 1, key_2016);

        // Log langkah
        log_2016.append("Langkah " + i_2016 + " : " + getNamaList_2016(data_2016) + "\n");
    }
}

// ===== SELECTION SORT =====
private void selectionSort_2016(ArrayList<Mahasiswa_2511532016> data_2016, StringBuilder log_2016) {
    log_2016.append("=== SELECTION SORT ===\n\n");
    int n_2016 = data_2016.size();

    for (int i_2016 = 0; i_2016 < n_2016; i_2016++) {
        // Cari indeks elemen terkecil dari bagian yang belum terurut
        int minIndex_2016 = i_2016;
        for (int j_2016 = i_2016 + 1; j_2016 < n_2016; j_2016++) {
            if (data_2016.get(j_2016).getNama_2016().compareToIgnoreCase(data_2016.get(minIndex_2016).getNama_2016()) < 0) {
                minIndex_2016 = j_2016;
            }
        }
        // Tukar elemen terkecil ke posisi i
        Mahasiswa_2511532016 temp_2016 = data_2016.get(i_2016);
        data_2016.set(i_2016, data_2016.get(minIndex_2016));
        data_2016.set(minIndex_2016, temp_2016);

        // Log pass
        log_2016.append("Pass " + (i_2016 + 1) + " : " + getNamaList_2016(data_2016) + "\n");
    }
}

// ===== BUBBLE SORT =====
private void bubbleSort_2016(ArrayList<Mahasiswa_2511532016> data_2016, StringBuilder log_2016) {
    log_2016.append("=== BUBBLE SORT ===\n\n");
    int n_2016 = data_2016.size();

    for (int i_2016 = 0; i_2016 < n_2016; i_2016++) {
        for (int j_2016 = 0; j_2016 < n_2016 - i_2016 - 1; j_2016++) {
            // Bandingkan dua elemen berdekatan
            if (data_2016.get(j_2016).getNama_2016().compareToIgnoreCase(data_2016.get(j_2016 + 1).getNama_2016()) > 0) {
                // Tukar jika urutan salah
                Mahasiswa_2511532016 temp_2016 = data_2016.get(j_2016);
                data_2016.set(j_2016, data_2016.get(j_2016 + 1));
                data_2016.set(j_2016 + 1, temp_2016);
            }
        }
        // Log pass
        log_2016.append("Pass " + (i_2016 + 1) + " : " + getNamaList_2016(data_2016) + "\n");
    }
}
```

Kode Program 3.9 – Method-Method Sorting

Ketiga method sorting; `insertionSort_2016()`, `selectionSort_2016()`, dan `bubbleSort_2016()` menggunakan logika yang identik dengan implementasi pada subbab sebelumnya. Perbedaan mendasarnya terletak pada tipe data yang diurutkan, yaitu String nama mahasiswa sebagai pengganti int, serta penggunaan method `compareToIgnoreCase()` sebagai pengganti operator `>` untuk membandingkan dua elemen.

Method `compareToIgnoreCase()` mengembalikan nilai positif jika string pertama secara alfabetis lebih besar dari string kedua, nilai negatif jika sebaliknya, dan nol jika keduanya sama. Sehingga dapat langsung digunakan sebagai kondisi penukaran elemen pada ketiga algoritma tersebut.

3.7 Method Konversi

```
// ===== HELPER: ambil list nama dari ArrayList =====
private String getNamaList_2016(ArrayList<Mahasiswa_2511532016> data_2016) {
    StringBuilder sb_2016 = new StringBuilder("");
    for (int i_2016 = 0; i_2016 < data_2016.size(); i_2016++) {
        sb_2016.append(data_2016.get(i_2016).getNama_2016());
        if (i_2016 < data_2016.size() - 1) sb_2016.append(", ");
    }
    sb_2016.append("]");
    return sb_2016.toString();
}
```

Kode Program 3.10 – Method untuk Konversi Data menjadi String

Method helper ini bertugas mengkonversi `ArrayList<Mahasiswa_2511532016>` menjadi String berformat [Andi, Budi, Charlie] untuk keperluan log visualisasi sorting. Menggunakan `StringBuilder` untuk efisiensi penggabungan String, dengan koma sebagai pemisah antar nama menggunakan kondisi `i < data.size() - 1` agar koma tidak muncul setelah elemen terakhir. Method ini dipanggil oleh ketiga method sorting setiap kali satu langkah atau pass selesai dieksekusi.

3.8 Method Reset

```
// ===== RESET =====
private void reset_2016() {
    dataMahasiswa_2016.clear();
    areaTabelData_2016.setText("");
    areaHasil_2016.setText("");
    fieldNama_2016.setText("");
    fieldNim_2016.setText("");
    fieldProdi_2016.setText("");
}
```

Kode Program 3.11 – Method untuk Mengulang Program

Method ini dipanggil saat tombol "Reset" ditekan, mengembalikan seluruh kondisi program ke keadaan awal. `dataMahasiswa_2016` dikosongkan menggunakan `clear()`, kedua `JTextArea` dikosongkan dengan `setText("")`, dan ketiga `TextField` input juga dikosongkan. Berbeda dengan `hapusData_2016()` yang hanya menghapus satu elemen terakhir, `reset_2016()` menghapus seluruh data sekaligus dan membersihkan semua tampilan — menjadikan program siap menerima input dari awal.

BAB IV

KESIMPULAN

Terdapat tiga algoritma sorting dasar yang dipelajari pada praktikum ini. Insertion Sort bekerja dengan cara mengambil satu elemen dan menyisipkannya ke posisi yang tepat di antara elemen-elemen yang sudah terurut sebelumnya. Selection Sort bekerja dengan cara mencari elemen terkecil dari bagian yang belum terurut lalu menempatkannya di posisi yang benar. Bubble Sort bekerja dengan cara membandingkan dua elemen yang berdekatan dan menukarnya jika urutannya salah, diulang hingga seluruh data terurut. Ketiga algoritma menghasilkan output yang sama namun dengan pendekatan yang berbeda.

Praktikum ini juga menunjukkan bahwa algoritma sorting tidak terbatas pada data bertipe numerik, melainkan dapat diterapkan pada data bertipe String menggunakan method `compareToIgnoreCase()` yang membandingkan dua String secara alfabetis tanpa membedakan huruf kapital dan huruf kecil. Hal ini diimplementasikan dalam studi kasus pengurutan nama mahasiswa secara ascending (A-Z).

Selain itu, praktikum ini memperkenalkan pembuatan aplikasi berbasis GUI menggunakan Java Swing, di mana proses sorting divisualisasikan secara langkah per langkah sehingga alur algoritma dapat dipahami secara intuitif. Penggunaan nested panel, JComboBox, JTextArea, dan JScrollPane menunjukkan bagaimana berbagai komponen Swing dapat dikombinasikan untuk membangun antarmuka yang fungsional. Secara keseluruhan, pemahaman terhadap algoritma sorting dasar memberikan fondasi yang kuat untuk mempelajari algoritma sorting yang lebih efisien seperti Merge Sort dan Quick Sort.

DAFTAR PUSTAKA

- [1]Oracle. *Java Documentation*. <https://docs.oracle.com/javase/>